```
# Practical: Comparative Analysis of Classification Algorithms
# Algorithms: SVM, Decision Tree, KNN, and Naive Bayes
# Dataset: Iris Flower Dataset

# ----------------------------------------------------
# IMPORT REQUIRED LIBRARIES
# ----------------------------------------------------

import pandas as pd

# Import the Iris Flower dataset
from sklearn.datasets import load_iris

# Import function to split the dataset
from sklearn.model_selection import train_test_split

# Import StandardScaler for feature scaling
from sklearn.preprocessing import StandardScaler

# Import make_pipeline to combine scaling and models
from sklearn.pipeline import make_pipeline

# Import classification algorithms
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

# Import evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    log_loss,
    confusion_matrix,
    classification_report
)

# ----------------------------------------------------
# STEP 1: LOAD THE DATASET
# ----------------------------------------------------

# Load the built-in Iris Flower dataset
data = load_iris()

# X contains input features and y contains target labels
X = data.data
y = data.target

# Display dataset information
print("Iris Flower Dataset loaded successfully.")
print("Number of samples:", X.shape[0])
print("Number of features:", X.shape[1])
print("Feature names:", data.feature_names)
print("Target classes:", data.target_names)

# ----------------------------------------------------
# STEP 2: SPLIT THE DATASET
# ----------------------------------------------------

# Use 80% of the data for training and 20% for testing
# random_state ensures reproducible results
# stratify preserves class proportions
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Display training and testing sample counts
print("\nTraining samples:", len(X_train))
```

```
print("Testing samples:", len(X_test))

# ------------------------------------------------
# STEP 3: INITIALIZE CLASSIFICATION MODELS
# ------------------------------------------------

models = {
    # SVM with RBF kernel and feature scaling
    "SVM": make_pipeline(
        StandardScaler(),
        SVC(
            kernel="rbf",
            probability=True,
            random_state=42
        )
    ),

    # Decision Tree with limited depth
    "Decision Tree": DecisionTreeClassifier(
        max_depth=5,
        random_state=42
    ),

    # KNN using the five nearest neighbors
    # Feature scaling is important for distance calculations
    "KNN": make_pipeline(
        StandardScaler(),
        KNeighborsClassifier(n_neighbors=5)
    ),

    # Gaussian Naive Bayes for numerical features
    "Naive Bayes": GaussianNB()
}

# Store the performance results of all models
results = []

# ------------------------------------------------
# STEP 4: TRAIN AND EVALUATE EACH MODEL
# ------------------------------------------------

for name, model in models.items():

    # Train the model
    model.fit(X_train, y_train)

    # Predict the flower species
    predictions = model.predict(X_test)

    # Predict class probabilities for log loss
    probabilities = model.predict_proba(X_test)

    # Calculate accuracy
    accuracy = accuracy_score(y_test, predictions)

    # Calculate weighted precision
    precision = precision_score(
        y_test,
        predictions,
        average="weighted",
        zero_division=0
    )

    # Calculate weighted recall
    recall = recall_score(
        y_test,
        predictions,
        average="weighted",
        zero_division=0
    )

    # Calculate weighted F1-score
    f1 = f1_score(
        y_test,
        predictions,
        average="weighted",
        zero_division=0
    )
```

```
    # Calculate log loss
    loss = log_loss(y_test, probabilities)

    # Save metrics for the current algorithm
    results.append({
        "Algorithm": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1,
        "Log Loss": loss
    })

    # Display evaluation results
    print("\n" + "=" * 50)
    print("Algorithm:", name)
    print("=" * 50)

    # Display class-wise performance
    print("\nClassification Report:")
    print(
        classification_report(
            y_test,
            predictions,
            labels=[0, 1, 2],
            target_names=data.target_names,
            zero_division=0
        )
    )

    # Display confusion matrix
    print("Confusion Matrix:")
    print(
        confusion_matrix(
            y_test,
            predictions,
            labels=[0, 1, 2]
        )
    )

# --------------------------------------------------
# STEP 5: COMPARE ALL CLASSIFICATION MODELS
# --------------------------------------------------

# Convert the results into a DataFrame
results_df = pd.DataFrame(results)

# Display the comparison table
print("\nCOMPARATIVE PERFORMANCE ANALYSIS")
print(results_df.round(4).to_string(index=False))

# --------------------------------------------------
# STEP 6: FIND THE BEST MODEL FOR EACH METRIC
# --------------------------------------------------

print("\nBEST MODEL FOR EACH METRIC")

# Higher values are better for these metrics
for metric in [
    "Accuracy",
    "Precision",
    "Recall",
    "F1-Score"
]:
    best = results_df.loc[results_df[metric].idxmax()]

    print(
        f"{metric}: {best['Algorithm']} "
        f"({best[metric]:.4f})"
    )

# Lower log loss is better
best_loss = results_df.loc[
    results_df["Log Loss"].idxmin()
]

```

```
print(
    f"Lowest Log Loss: {best_loss['Algorithm']} "
    f"({best_loss['Log Loss']:.4f})"
)

# ------------------------------------------------
# STEP 7: SAVE THE RESULTS
# ------------------------------------------------

# Save the comparison table to a CSV file
results_df.to_csv(
    "comparison_results.csv",
    index=False
)

print("\nResults saved to comparison_results.csv")
print("Iris classification analysis completed successfully.")

```

```
Number of features: 4
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target classes: ['setosa' 'versicolor' 'virginica']

Training samples: 120
Testing samples: 30

==================================================
Algorithm: SVM
==================================================

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.90      0.95        10
   virginica       0.91      1.00      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]

==================================================
Algorithm: Decision Tree
==================================================

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10

    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

==================================================
Algorithm: KNN
==================================================

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.83      1.00      0.91        10
   virginica       1.00      0.80      0.89        10

    accuracy                           0.93        30
```